

# CO1 ASSESSMENT TOOL 1 – PSEUDOCODE WRITING

## CSA0206 – C Programming

**Student Name:** Rakshana AR    **Register No.:** 1921260027

### 1. Inputs

- Consumer ID
- Units consumed
- Applicable tariff value (per unit)
- Surcharge threshold
- Surcharge percentage

### 2. Outputs

- Consumer ID
- Units consumed
- Energy charge
- Surcharge
- Total payable amount

### 3. Constraints

- Units consumed, tariff, threshold and surcharge percentage must be non-negative; the program validates this after input and stops with an error message otherwise.
- The fixed service charge is defined once as a constant (preprocessor macro).
- The surcharge is applied only when the preliminary bill (energy charge + service charge) exceeds the threshold; otherwise it is zero.

### 4. Pseudocode

```
BEGIN
    DEFINE SERVICE_CHARGE AS CONSTANT = 100.00
    DECLARE consumerID AS INTEGER
    DECLARE units, tariff, threshold, surchargeRate AS REAL
    DECLARE energyCharge, surcharge AS REAL

    // ---- INPUT ----
    INPUT consumerID, units, tariff, threshold, surchargeRate

    // ---- VALIDATION ----
    IF units < 0 OR tariff < 0 OR threshold < 0 OR surchargeRate < 0 THEN
        OUTPUT "Error: values must be non-negative"
        STOP
    END IF

    // ---- CALCULATION + DECISION ----
    energyCharge <- units * tariff
    surcharge <- (energyCharge + SERVICE_CHARGE > threshold)
                  ? (energyCharge + SERVICE_CHARGE) * surchargeRate / 100
                  : 0

    // ---- OUTPUT ----
    OUTPUT consumerID
    OUTPUT units
    OUTPUT energyCharge
    OUTPUT surcharge
```

```

        OUTPUT energyCharge + SERVICE_CHARGE + surcharge    // total payable
    END

```

## 5. Corresponding C Program

```

#include <stdio.h>
#define SERVICE_CHARGE 100.00

int main()
{
    int consumerID;
    double units, tariff, threshold, surchargeRate;
    double energyCharge, surcharge;

    printf("Enter Consumer ID: ");
    scanf("%d", &consumerID);
    printf("Enter Units Consumed: ");
    scanf("%lf", &units);
    printf("Enter Applicable Tariff: ");
    scanf("%lf", &tariff);
    printf("Enter Surcharge Threshold: ");
    scanf("%lf", &threshold);
    printf("Enter Surcharge Percentage: ");
    scanf("%lf", &surchargeRate);

    if (units < 0 || tariff < 0 || threshold < 0 || surchargeRate < 0)
    {
        printf("Error: values must be non-negative.\n");
        return 1;
    }

    energyCharge = units * tariff;
    surcharge = (energyCharge + SERVICE_CHARGE > threshold)
        ? (energyCharge + SERVICE_CHARGE) * surchargeRate / 100
        : 0;

    printf("\n--- ELECTRICITY BILL ---\n");
    printf("Consumer ID      : %d\n", consumerID);
    printf("Units Consumed   : %.2lf\n", units);
    printf("Energy Charge    : %.2lf\n", energyCharge);
    printf("Surcharge        : %.2lf\n", surcharge);
    printf("Total Payable    : %.2lf\n", energyCharge + SERVICE_CHARGE + surcharge);
    return 0;
}

```

## 6. Justification

**Data types.** `int` is used for `consumerID` because it is a whole-number identifier on which no arithmetic is performed. `double` is used for `units`, `tariff`, `threshold`, `surcharge` percentage, `energy` charge and `surcharge` because they can hold decimal values and `double` gives enough precision for money calculations. The service charge is a `#define` constant, so it cannot be changed accidentally and is edited in one place.

**Operators.** Arithmetic: `*` (`units × tariff`, and the percentage multiplication), `/` (divide by 100 for the percentage) and `+` (add the service charge and surcharge). Relational: `>` compares the preliminary bill with the threshold, and `<` is used in validation. Conditional: `?:` selects the surcharge amount or 0 in a single expression, so no separate preliminary-bill, total-bill or if/else variables are needed. Logical `||` combines the validation checks, and `=` is assignment.

**Precedence.** In `energyCharge + SERVICE_CHARGE > threshold`, `+` has higher precedence than `>`, so the preliminary bill is computed first and then compared with the threshold. In `(energyCharge + SERVICE_CHARGE) * surchargeRate / 100`, the parentheses force the addition to happen before the multiplication; without them, `*` would bind tighter and only `SERVICE_CHARGE` would be multiplied. The conditional operator has lower precedence than both arithmetic and relational operators, so the condition needs no extra grouping, and it has higher precedence than `=`, so the whole conditional result is assigned to `surcharge`. In the final output expression, `+` is evaluated before the value is passed to `printf`.

**Associativity.** `*` and `/` have equal precedence and associate left to right, so `x * surchargeRate / 100` is evaluated as `(x * surchargeRate) / 100`, which is the intended percentage calculation. `+` also associates left to right, so `energyCharge + SERVICE_CHARGE + surcharge` is `(energyCharge + SERVICE_CHARGE) + surcharge`. The conditional operator `?:` and the assignment operator `=` associate right to left, so the conditional expression is completely evaluated before its result is stored in `surcharge`.

### Note

The question does not give numerical values for the tariff, threshold or surcharge rate, so these are taken as inputs. The fixed service charge is represented by a constant (100.00 is used as a sample value and can be changed). Sample run: units = 300, tariff = 6, threshold = 1500, surcharge = 10% gives energy charge 1800.00, surcharge 190.00 and total payable 2090.00.